

Splitting Large Convolutional Neural Network Layers to Run Real-Time Applications on Mixed-Reality Hardware: Extended Abstract

Anthony Beug*
University of Regina

Howard J. Hamilton†
University of Regina

ABSTRACT

When executing computationally expensive Convolutional Neural Networks (CNNs) in real-time mixed-reality applications, some convolutional layers may take longer than the target frame time to execute. Here, dropped frames produced by large convolutional layers are avoided by dividing the work performed in a convolution so that it can be executed over multiple frames. A novel method is described to schedule the execution of the layers of a CNN by modifying the model graph of a pretrained CNN by splitting large convolutions. Overhead is introduced, but the average frame rate is increased since delays produced by computing large layers are avoided.

Keywords: Convolutional neural network, mixed-reality, augmented-reality, real-time, object detection, scheduling.

Index Terms: Computing methodologies—Graphics systems and Interfaces—Mixed / augmented reality; Computing methodologies—Machine learning—Machine learning approaches—Neural networks

1 INTRODUCTION

Executing a Convolutional Neural Network (CNN) on current mixed-reality (MR) hardware, such as the Microsoft HoloLens 2, can be computationally demanding. In a real-time MR application that must maintain a target frame rate while executing an expensive CNN, some of the convolutional layers may take longer than the target frame time to execute. To achieve a target frame rate for graphics, only a certain amount of time can be spent executing a neural network or part of a neural network between frames. If a neural network is executed on a GPU in an MR application, it must not overutilize GPU resources, because the GPU may also be used for other MR tasks such as rendering. To avoid exceeding the target frame time, the work performed in a large convolutional layer can be split into multiple parts and executed over multiple frames.

This work presents a method to use existing convolution splitting techniques to split large convolutions that take too long to execute in a single frame. Output is consistent after splitting, allowing the technique to be applied to any existing pretrained CNN. This work also presents a method to manually design a static schedule for a target MR device to execute the CNN layers, including split convolutional layers, across multiple frames to maintain application performance while executing a long-running CNN.

2 SPLITTING CONVOLUTIONS

To execute large convolutions over multiple frames, work must be divided over multiple frames. Two main convolution splitting techniques are used for this purpose. Jin and Hong [1] propose

Split-CNN, which spatially splits convolution input into rectangular patches that can be executed separately. The splitting can be performed for a single layer, or in a multi-layer approach where multiple convolutional layers are performed before combining the resulting output patches. Here the single-layer approach is used to avoid changing the output of the model so that any pretrained CNN can be used. Alternatively, convolutions can also be split along their output filters, as demonstrated in Xception [2] and ResNeXt [3]. Fig. 1 shows an example convolution split spatially and along output filters into 4 sub-convolutions.

As a novel contribution, this work shows how to modify the model graph of a pretrained CNN to split its large convolutions. By using existing splitting techniques in this new context, we derive a schedule that avoids exceeding the target frame time without needing to retrain the CNN. After identifying large convolutions, the convolution nodes in the graph are split and replaced with multiple sub-convolutions nodes. Their outputs are concatenated to produce the output of the original convolution.

To split a convolution along the spatial dimensions, the original convolution input is sliced along the spatial dimensions into different blocks, which are used as input to each sub-convolution. To avoid inserting a pad operator to the graph, padding is performed in the sub-convolution nodes. The first and last sub-convolution along a spatial axis have starting and ending pads, respectively. Sub-convolutions in the middle contain no padding because their inputs are sliced with an overlapping portion. With this approach, the same filter weights are used for each sub-convolution. To split a convolution along its output filters, the filter weights and biases are sliced into groups along the channel axis. Each sub-convolution uses one of the sliced groups and all use the same input tensor. Grouped convolutions can be split by splitting the weights, biases, and input along the channel axis to divide the number of groups performed in the resulting sub-convolutions.

When splitting a convolution into sub-convolutions and concatenating the results, batch normalization [4] (BN) fusion

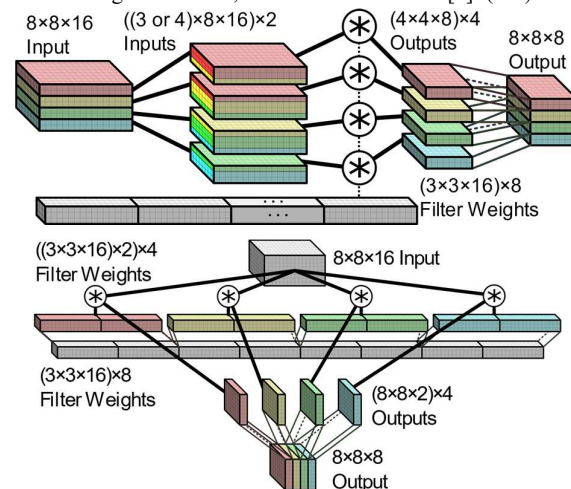


Figure 1: (Top) Splitting a convolution the along spatial height dimension. (Bottom) Splitting a convolution along output filters.

*e-mail: anthony.beug@uregina.ca

†e-mail: hamilton@cs.uregina.ca

cannot be applied if the BN layer occurs after the concatenation layer. For a convolution that is split along its output filters, BN inputs must also be split along the corresponding channels. The means μ , variances σ^2 , scales γ and biases β tensors are split into smaller tensors: i.e., $\{\mu_0, \mu_1 \dots \mu_S\}$, $\{\sigma_0^2, \sigma_1^2 \dots \sigma_S^2\}$, $\{\gamma_0, \gamma_1 \dots \gamma_S\}$, $\{\beta_0, \beta_1 \dots \beta_S\}$ for S splits to match the splitting applied to the sub-convolution. For a spatially split convolution, the BN layer can be copied and applied to the output of the sub-convolutions, because μ, σ^2, γ and β are the same for each sub-convolution.

3 LAYER SCHEDULING

Profiling tools are used to measure the average execution time of each CNN layer on a target device; they reveal convolutions that take too long to execute in a frame. To maintain real-time application performance, no layer should take more than the frame time budget b , the maximum time that can be spent executing the neural network on the frame. A value for b can be set with respect to the target frame time and average GPU time of other GPU tasks such as rendering. Any convolutional layer that takes longer than b should be split such each sub-convolution takes less than b .

By considering the model graph, which shows the dependencies between the layers of a CNN, a schedule is created manually by treating the profiled layer times as time blocks. Layers are grouped together to be executed on the same frame and each group must fit in the target frame time budget. The schedule attempts to satisfy $\sum L_i^f < b$, while maximizing $\sum L_i^f$, where L_i^f is the time to execute i^{th} layer in frame f . This assumes b remains constant and the time to execute a specific layer is constant. The actual timings for these values may fluctuate at runtime and cause dropped frames, so this is not a perfect method. However, the schedule can be modified after profiling the scheduled layers. The schedule designed is a static schedule, meaning that the same layers are executed on their corresponding frames each time the CNN is executed.

In practice, the layers cannot be split to perfectly fill the frame time budget, because if the convolutions are split into blocks that are too small, they may underutilize the GPU. If a convolution can be further split to fill part of a gap in the frame time budget without over-splitting, it should be split. If a convolution splitting produces two or more sub-convolutions that execute on the same frame, the convolution should instead be split non-uniformly such that only one sub-convolution runs on a given frame. These ideas can be used to create a static schedule for the CNN layers.

4 EXPERIMENT

To evaluate the splitting and scheduling techniques, several CNNs that were too expensive to run on a HoloLens 2 device in real time were tested. The experimental environment used a HoloLens 2 application built in the Unity game engine. CNN execution was performed on the GPU with the Unity Barracuda [5] inference engine. The Unity profiler was used to measure layer execution time of the compute shaders that run each layer. These times were used to manually design static execution schedules using a frame time budget of 13 ms. YOLOv4-Tiny [6] with a 416x416 resolution was tested as well as EfficientNet-Lite4 and ResNet50-V2 image classification CNNs from ONNX model zoo [7]. Convolutions earlier in the CNNs have larger spatial dimensions than the number of output filters so they are split spatially and vice versa. For some frames in the schedule, b is not fully used since the layers cannot be split perfectly. Thus, time is wasted waiting for VSync.

5 RESULTS

Results are summarized in Table 1. To measure the added overhead of splitting the convolutions, the single-frame GPU execution times were obtained for the unsplit and split CNNs. An increase was expected due to splitting overhead, and the addition of slice and

Table 1: Performance stats for unsplit and split CNNs.

	GPU Time 1-Frame	Schedule CPU Time	FPS 1-Frame	FPS
YoloV4-Tiny	413.42 ms	557.00 ms	12.36	32.56
↳ Split	442.29 ms	732.22 ms	11.72	53.54
EfficientNet-Lite4	311.57 ms	605.81 ms	14.86	53.89
↳ Split	319.24 ms	535.62 ms	14.26	57.81
ResNet50-V2	529.52 ms	995.83 ms	10.95	53.62
↳ Split	551.49 ms	912.19 ms	9.18	58.15

concatenation layers. For YOLOv4-Tiny, the CPU schedule time is increased after splitting due to the inability to perfectly split the large convolutions that persist over multiple frames. However, most of the frames that exceed b are avoided. The other 2 CNNs have no convolutions that take more than 2 frames to execute, but by splitting convolutions so they can execute at the end of one frame and the start of the next frame to fill gaps, faster schedule times were obtained. Application frame rate was increased after using the split schedules. After splitting, model accuracy should remain identical because concatenating the outputs of sub-convolutions is equivalent to performing the original convolution.

6 CONCLUSION

By splitting large convolutions that are too costly to execute in a single frame and designing a static schedule to execute CNN layers on different frames, frame rate can be increased while running expensive CNNs on real-time MR applications. Frame rate is increased at the cost of total execution time. Admittedly, a large increase in total execution time was produced, but a higher frame rate is arguably more important in an MR application.

The static scheduling approach used in this paper has weaknesses such as time-consuming profiling and hardware dependence, which could be resolved by designing a dynamic scheduling system that attempts to determine a schedule for CNN layers at runtime.

ACKNOWLEDGMENTS

We thank: ISM Canada and Natural Sciences and Engineering Research Council of Canada (grants 514906-17 and 2019-07301).

REFERENCES

- [1] T. Jin and S. Hong, "Split-CNN: Splitting Window-based Operations in Convolutional Neural Networks for Memory System Optimization," in *Proc. of the 24th Int. Conf. on Architectural Support for Program. Lang. and Operating Syst. (ASPLOS)*, 2019, pp. 835-847.
- [2] F. Chollet, "Xception: Deep Learning with Depthwise Separable Convolutions," 2017. [Online]. Available: <https://arxiv.org/abs/1610.02357>.
- [3] S. Xie, R. Girshick, P. Dollár, Z. Tu and K. He, Aggregated Residual Transformations for Deep Neural Networks. 2016. [Online]. Available: <https://arxiv.org/abs/1611.05431>.
- [4] S. Ioffe and C. Szegedy, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift," in *Proc. of the 32nd Int. Conf. on Machine Learning (ICML)*, 2015, vol. 37, pp. 448-456.
- [5] "Unity Barracuda," Unity Technologies, 2019. [Online]. Available: <https://github.com/Unity-Technologies/barracuda-release>.
- [6] C.-Y. Yang, A. Bochkovskiy and H.-Y. M. Liao, "Scaled-YOLOv4: Scaling Cross Stage Partial Network," 2021. [Online]. Available: <https://arxiv.org/abs/2011.08036>.
- [7] ONNX Model Zoo. The Linux Foundation, 2020. [Online]. Available: <https://github.com/onnx/models>